SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	T Jyothi Reddy	**Reg. No.:**	192472079
Section / Batch:	2024	**Date:**	

Code of Conduct

I ________T Jyothi Reddy (192472079)__________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___T Jyothi Reddy______________________

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for "The cat drinks milk"

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	20	20	20	20	8	88	88

Faculty In-charge	Course Coordinator	Program Director
Dr.T. Kumaragurubaran		
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

Program:

```
# HMM POS TAGGING USING VITERBI

corpus = [
    ["The/DT","boy/NN","eats/VBZ","rice/NN"],
    ["The/DT","girl/NN","drinks/VBZ","milk/NN"],
    ["A/DT","cat/NN","drinks/VBZ","milk/NN"],
```

```
    ["The/DT","dog/NN","chases/VBZ","cat/NN"],
    ["A/DT","teacher/NN","teaches/VBZ","students/NNS"],
    ["Students/NNS","study/VBP","English/NN"],
    ["Birds/NNS","fly/VBP","high/RB"],
    ["Children/NNS","play/VBP","games/NNS"]
]

tags = {}
words = {}
trans = {}
start = {}

# Separate words and tags + count
for s in corpus:
    prev = ""
    for x in s:
        w,t = x.split("/")
        tags[t] = 1

        if t not in words:
            words[t] = {}
        words[t][w] = words[t].get(w,0) + 1

        if prev:
            if prev not in trans:
                trans[prev] = {}
            trans[prev][t] = trans[prev].get(t,0) + 1

        prev = t

    start[s[0].split("/")[1]] = start.get(s[0].split("/")[1],0) + 1


# Emission probability
emit = {}

for t in words:
    emit[t] = {}
    total = sum(words[t].values())

    for w in words[t]:
        emit[t][w] = words[t][w] / total


# Transition probability
transition = {}

for t in trans:
    transition[t] = {}
    total = sum(trans[t].values())

    for x in trans[t]:
        transition[t][x] = trans[t][x] / total


# Viterbi
```

```
def viterbi(sentence):

    dp = [{}]
    path = [{}]

    for t in tags:
        e = emit[t].get(sentence[0],0)
        st = start.get(t,0) / len(corpus)
        dp[0][t] = st * e
        path[0][t] = ""

    for i in range(1,len(sentence)):
        dp.append({})
        path.append({})

        for t in tags:
            best = 0
            prev_best = ""

            for p in tags:
                prob = dp[i-1][p] * transition.get(p,{}).get(t,0)
                prob *= emit[t].get(sentence[i],0)

                if prob > best:
                    best = prob
                    prev_best = p

            dp[i][t] = best
            path[i][t] = prev_best

    last = max(dp[-1], key=dp[-1].get)

    result = [last]

    for i in range(len(sentence)-1,0,-1):
        last = path[i][last]
        result.insert(0,last)

    return result


# Prediction
sentence = "The cat drinks milk".split()
result = viterbi(sentence)

print("Input:", " ".join(sentence))

for i in range(len(sentence)):
    print(sentence[i], "->", result[i])

print("POS Tags:", result)
```
Output:

```
PS C:\Users\RAJASRI\OneDrive\文档\code conversion system>  c:; cd 'c:\Users\RAJASRI\OneDrive\文档\code conversion system'; & 'C:\Users\RAJASRI\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\RAJASRI\.vscode\extensions\ms-python.debugpy-2026.7.12111009-win32-x64\bundled\libs\debugpy\launcher' '49223' '--' 'C:\Users\RAJASRI\Downloads\POS Tgging.py'
Input: The cat drinks milk
The -> DT
cat -> NN
drinks -> VBZ
milk -> NN
POS Tags: ['DT', 'NN', 'VBZ', 'NN']
PS C:\Users\RAJASRI\OneDrive\文档\code conversion system>
```